# ADVANCED PYTHON PROGRAMMING

**NAME: ANEESH J V R**

**REG NO. : 23MID0306**

**Project Writeup**

# Movie Analytics Dashboard

## 1. Title

Movie Analytics Dashboard: A Full-Stack Web Application for Interactive Movie Data Analysis

## 2. Motivation

The motivation behind this project is to develop a web-based application that transforms raw movie data into meaningful insights through interactive visualizations, statistical analysis, and filtering.

The movie industry generates large amounts of data related to genres, ratings, popularity, release dates, budgets, and revenue. Analyzing this information manually using spreadsheets or raw CSV files can be time-consuming and difficult, particularly when comparing trends across thousands of movies.

The Movie Analytics Dashboard addresses this challenge by providing a centralized platform where users can explore movie datasets, identify patterns, and make data-driven observations.

The main objectives are:

- To develop a full-stack web application for movie data analysis.
- To process and analyze a large movie dataset using Python.
- To present movie statistics through interactive charts, tables, and filters.
- To enable users to explore genre distributions, rating patterns, and release-year trends.
- To provide structured API endpoints for accessing analytical results.
- To demonstrate the integration of frontend development, backend processing, database management, and data analytics.

## 3. Existing Gap, Motivation, and USP

### Existing Gap

Movie datasets are often available as raw CSV files or through websites that provide limited analytical functionality. Users who want to perform customized comparisons may need to download datasets and manually process them using programming tools.

Some limitations of existing approaches include:

- Difficulty analyzing thousands of movie records efficiently.
- Limited flexibility in comparing genres, ratings, and release years.
- Lack of a unified interface for visual exploration and statistical analysis.
- Difficulty accessing processed movie information programmatically.
- Separation of data processing, visualization, and API access across different tools.

### Proposed Solution

The Movie Analytics Dashboard provides an integrated platform that loads movie datasets, processes the information using Python, stores structured records in a database, and presents analytical results through a web interface.

The application also provides API endpoints that allow external applications or partners to retrieve selected movie analytics.

### Unique Selling Proposition (USP)

The key USP is the integration of movie data processing, database-backed storage, interactive visualization, and API-based data access within a single application.

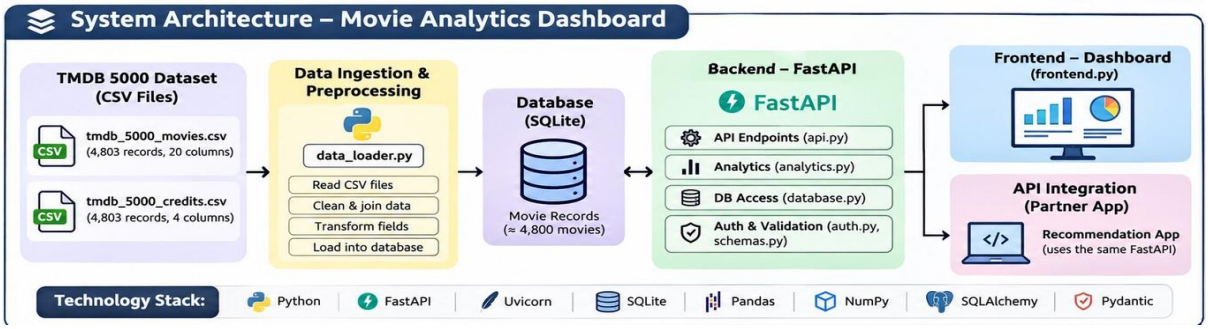
The main features include:

- Interactive movie analytics dashboard.
- Genre, rating, and release-year analysis.
- Search and filtering capabilities.
- Statistical summaries and visualizations.
- Persistent movie data storage.
- REST API access to analytical information.
- Authentication and schema validation components.
- Potential for deployment and integration with external applications.

# 4. Architecture

The application follows a full-stack architecture consisting of a frontend, backend API, analytics layer, and database.

### Architecture Diagram



### Architecture Components

1. Frontend Layer

The frontend provides the dashboard interface through which users view charts, tables, movie statistics, and filtering controls. It communicates with the backend using HTTP requests.

2. Backend Layer

The backend is implemented using FastAPI. It handles incoming requests, coordinates analytics operations, and returns structured responses through REST API endpoints.

3. Analytics Layer

The analytics module processes movie records and calculates summary statistics, genre distributions, rating comparisons, and release-year trends.

4. Database Layer

The database layer uses SQLite for local persistence. Database models and connection management are handled through `models.py` and `database.py`.

5. Data Ingestion Layer

The data loader imports the TMDB CSV files, processes the records, and prepares the data for database storage and analytics.

6. External Integration Layer

The application includes API-related components that support external data access and partner integration.

# 5. Complete Technology Stack

| Technology | Purpose |
|---|---|
| Python | Core programming language for backend development and data analysis. |
| FastAPI | Backend framework for building REST APIs. |
| HTML | Structure of the web interface. |
| CSS | Styling and layout of the dashboard. |
| JavaScript | Frontend interactions and asynchronous API requests. |
| Pandas | Data cleaning, transformation, and analysis. |
| NumPy | Numerical operations and statistical calculations. |
| SQLite | Local relational database for persistent movie records. |
| SQLAlchemy | Database models and database interaction. |
| Pydantic | Data validation and API request/response schemas. |
| Uvicorn | ASGI server for running the FastAPI application. |

| | |
|---|---|
| Pytest | Automated testing of application components. |
| Git and GitHub | Version control and project repository management. |
| Render | Deployment configuration and hosting support. |

### Main Project Files

| File | Responsibility |
|---|---|
| `main.py` | Application entry point and server configuration. |
| `frontend.py` | Dashboard interface and frontend behavior. |
| `api.py` | API routes and request handling. |
| `analytics.py` | Analytical calculations and data summaries. |
| `database.py` | Database connection and session management. |
| `models.py` | Database models and table definitions. |
| `data_loader.py` | CSV loading, cleaning, and data preparation. |
| `auth.py` | Authentication-related functionality. |
| `schemas.py` | Request and response validation schemas. |
| `partner_client.py` | Partner API integration functionality. |
| `config.py` | Application configuration. |
| `tests/` | Automated tests for application functionality. |

# 6. Datasets Used

The project uses the TMDB 5000 Movie Dataset, which contains movie metadata and credits information.

### Dataset Files

| Dataset | Description |
|---|---|
| `tmdb_5000_movies.csv` | Contains movie metadata, including titles, genres, release dates, ratings, popularity, and financial information where available. |
| `tmdb_5000_credits.csv` | Contains cast and crew information associated with movies. |

The two datasets are joined using the movie identifier.

### Data Processing

The data processing workflow consists of:

1. Loading the CSV files using Pandas.
2. Cleaning and preparing the movie records.
3. Joining the movie and credits datasets using the movie ID.
4. Converting relevant fields into structured records.
5. Storing processed movie data in the database.
6. Using the processed data for analytics and visualization.

The application processes approximately 4,800 movie records from the supplied dataset.

Dataset source: Kaggle – TMDB 5000 Movie Dataset.

https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

# 7. Prompts Used for Website and Application Construction

MASTER PROMPT — TRANSFORM MY EXISTING MOVIE ANALYTICS DASHBOARD INTO A PREMIUM, SHARED MOVIE INTELLIGENCE PLATFORM

Act as a senior full-stack Python engineer, database architect, product designer, data analyst, API engineer, and technical mentor.

I am uploading my existing Movie Analytics Dashboard project. Your task is to upgrade and improve this project substantially while preserving working functionality.

Do not simply generate a new generic dashboard. Inspect the existing source code, identify what already works, and modify the existing project.

I need a polished, attractive, functional movie analytics application that can be used in my college final review. My partner is separately building a Movie Discovery and Recommendation System. Both applications must share the same live movie data through a common API and a persistent database.

The result must be more than a visualization of a static CSV. It should function as a movie intelligence platform with real analytics, movie discovery, editable metadata, shared user interactions, actual poster images, and live data exchange.

**1. MY PROJECT AND MY PARTNER'S PROJECT**

My project:
**Movie Analytics Dashboard**

My partner's project:
**Movie Discovery and Recommendation System**

Our common source dataset:

- tmdb_5000_movies.csv
- tmdb_5000_credits.csv

The files belong to the TMDB 5000 Movie Dataset.

Inspect the actual uploaded files and the existing application before making assumptions.

My project should focus on:

- Movie industry analytics.
- Financial performance.
- Genre analysis.
- Ratings and popularity.
- Release trends.
- Production insights.
- Movie comparisons.
- User engagement analytics.
- Shared movie metadata management.

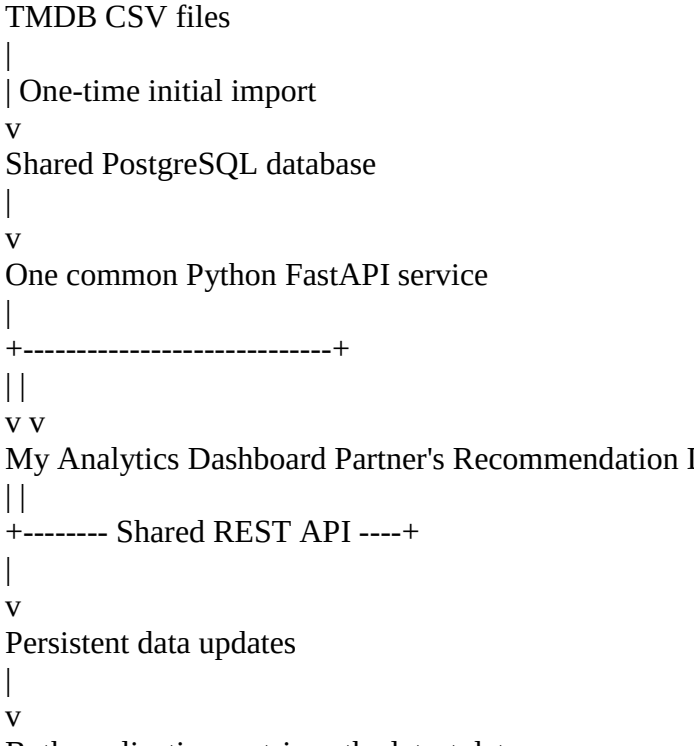
My partner's project should focus on:

- Movie recommendations.
- Similar movies.
- Genre-based discovery.
- Cast/director-based discovery.
- User preferences.
- Personal ratings.
- Likes and watchlists.
- Recommendation feedback.

Keep the two applications separate, but make them work with the same underlying data.

#### 2. THE ARCHITECTURE I WANT

Use this architecture:




Preferred infrastructure:

- PostgreSQL hosted on Supabase.
- Python FastAPI service hosted on Render.
- Pandas and NumPy for analytical processing.
- SQLAlchemy for database access.
- HTML/CSS/JavaScript embedded in Python for the frontend.
- Chart.js for charts.
- TMDB API for poster-path enrichment, subject to its terms.

Do not assume the cloud database or hosting account already exists. Implement and test the local version first, then provide exact cloud setup instructions.

Both applications must be able to use the same deployed API base URL.

Do not create separate databases for the two applications.

Do not rely on two local CSV files as the authoritative live dataset.

#### 3. FIRST INSPECT MY EXISTING PROJECT

The uploaded project already includes modules such as:

- main.py
- api.py
- analytics.py
- data_loader.py
- frontend.py
- database.py
- models.py
- schemas.py
- auth.py
- config.py
- manage.py
- tests/

The existing application uses FastAPI, SQLAlchemy, Pandas, and a database-backed movie catalogue.

Inspect the existing API routes, database schema, analytics functions, authentication, frontend, and tests.

Reuse and improve working components. Do not discard functioning code unnecessarily.

Identify:

1. Existing features that already work.
2. Existing features that are incomplete.
3. Features that are missing.
4. Database and API changes required.
5. UI improvements required.
6. Tests needed to prove the new behavior.

Then implement the changes. Do not stop after producing an analysis or plan.

#### 4. MAKE THE FRONTEND ATTRACTIVE AND PROFESSIONAL

The current website feels too basic. Redesign it as a premium movie intelligence application.

I will provide a reference mockup. Use it as the visual foundation, but improve the information architecture, interactions, and usability.

## Design direction

Use a cinematic dark-navy sidebar, a light dashboard workspace, blue and violet accents, refined typography, polished cards, clear chart labels, subtle shadows, responsive layouts, and attractive movie poster imagery.

The result should feel like a professional analytics product rather than a college demo made from generic charts.

Implement the following main navigation:

1. Overview
2. Movie Explorer
3. Industry Analytics
4. Financial Insights
5. Genre Intelligence
6. Ratings and Popularity
7. Production Insights
8. User Engagement
9. Data Management

Provide a global search, useful filters, clear page titles, loading states, empty states, and helpful error messages.

Keep the interface responsive.

Do not leave non-functional navigation buttons.

#### 5. OVERVIEW DASHBOARD

Create a visually impressive overview page.

Use KPI cards for:

- Total movies.
- Average dataset rating.
- Total genres.
- Total recorded revenue.
- Average popularity.
- Movies released in the selected period.

Calculate all metrics from the live database.

Add visualizations for:

- Movie releases by year.
- Genre distribution.
- Rating distribution.
- Revenue trends.
- Budget versus revenue.
- Popularity versus ratings.
- Highest-revenue movies.
- Highest-rated movies.

Add a time-period filter where meaningful.

Use interactive charts with tooltips, readable labels, legends, and sensible formatting.

Include a "Last updated" indicator based on actual data timestamps.

Do not hard-code KPI values or chart data.

Do not use meaningless charts just to fill the screen. Each visualization should answer a useful analytical question.

#### 6. INDUSTRY AND FINANCIAL ANALYTICS

Use the available dataset fields to implement meaningful analytics.

## Financial insights

- Highest-grossing movies.
- Highest-budget movies.
- Budget versus revenue scatter plot.
- Estimated profit.
- Estimated ROI.
- Revenue by genre.
- Revenue by year.
- Budget and revenue distributions.

Define calculations carefully.

Estimated profit = revenue - budget.

Estimated ROI (%) = ((revenue - budget) / budget) × 100.

Only calculate ROI for valid, positive budgets and valid revenue data.

Clearly label these as simplified estimates, not audited financial results. Do not treat missing budget or revenue values as genuine zeroes.

## Genre intelligence

- Movie count per genre.
- Average rating per genre.
- Revenue per genre.
- Popularity by genre.
- Genre comparison.
- Multi-genre movie handling.

A movie may have several genres, so document how the counts and aggregates work.

## Rating and popularity

- Rating distribution.
- Top-rated movies.
- Vote-count comparisons.
- Rating versus popularity.
- Minimum-vote threshold.
- Popularity rankings.

Make the ranking rules explicit. A high rating with very few votes should not automatically dominate every ranking.

## Production insights

- Movies by original language.
- Production companies with the most movies.
- Production-country distribution.
- Director statistics where the data permits.

## Movie comparisons

Allow users to select two or more movies and compare their:

- Ratings.
- Vote counts.
- Popularity.
- Budget.
- Revenue.
- Runtime.
- Genres.
- Release year.

Present comparisons with attractive visual elements and a concise summary of the differences.

#### 7. MOVIE EXPLORER WITH REAL POSTERS

I want actual movie posters, not only plain text cards or random placeholder images.

The existing TMDB CSV files may not contain poster paths. Inspect the actual schema.

If poster paths are missing, implement an optional poster-enrichment process using the official TMDB API.

Requirements:

- Read the movie ID and title.
- Match the record using the TMDB movie ID whenever possible.
- Retrieve the corresponding poster path from a reliable source.
- Store the poster path in the shared database.
- Cache the path so the same record is not repeatedly enriched.
- Do not fetch all posters on every page load.
- Avoid mismatching posters to movies with similar titles.
- Provide a fallback when a poster cannot be found.
- Do not expose TMDB API credentials in frontend JavaScript.
- Respect the applicable TMDB API usage and attribution requirements.

Make the movie explorer visually rich, with poster cards showing:

- Poster.
- Title.
- Release year.
- Rating.
- Genre.
- Popularity.
- View details.
- Add to comparison.
- Edit metadata, if the user is authorized.

Implement search and filters for:

- Movie title.
- Genre.
- Year range.
- Minimum rating.
- Minimum vote count.
- Language.
- Sorting by rating, popularity, revenue, or year.

Use pagination or an appropriate result limit.

#### 8. SHARED DATABASE: SINGLE SOURCE OF TRUTH

Use one shared PostgreSQL database for the canonical movie catalogue.

The database must preserve the fields needed for both applications.

At minimum, support:

- TMDB movie ID.
- Title.
- Overview.
- Release date.
- Genres.
- Rating.
- Vote count.
- Popularity.
- Budget.
- Revenue.
- Runtime.
- Original language.
- Keywords.
- Production companies.
- Production countries.
- Director.
- Cast.
- Crew.
- Poster path.
- Created timestamp.
- Updated timestamp.
- Last editor.

Preserve the TMDB movie ID as the stable movie identifier.

Store nested metadata safely using appropriate JSONB columns or normalized related tables.

The original CSV files should be treated as the initial import and backup. They must not overwrite edits automatically during application startup.

Create a safe, repeatable import process that does not duplicate records or destroy edits.

The database, not an in-memory DataFrame or the original CSV, must be the authoritative source for all edits.

#### 9. DATA EXCHANGE BETWEEN THE TWO APPLICATIONS

This is one of the most important requirements.

My application and my partner's application must use the same shared API.

Document the API contract so my partner can independently implement their recommendation system.

At minimum, support endpoints equivalent to:

GET /api/health

GET /api/movies

GET /api/movies/{movie_id}

GET /api/movies/search

PATCH /api/movies/{movie_id}

GET /api/analytics/summary

GET /api/analytics/genres

GET /api/analytics/years

GET /api/analytics/financials

GET /api/export/movies.csv

Add interaction endpoints described in the next section.

All reads must return current database values.

All writes must be validated and committed to the shared database.

Both applications must use the same movie ID and the same field definitions.

Document request schemas, response schemas, filtering, pagination, authentication, and error responses.

**10. SHARED USER INTERACTIONS**

I want the applications to exchange meaningful information, not only movie metadata.

Create a separate database table for user interactions. Do not overwrite TMDB's aggregate vote average when a user submits a personal rating.

Use a model such as:

UserInteraction:

- id.
- user_id or anonymous session identifier.
- movie_id.
- interaction_type.
- rating, when applicable.
- created_at.
- updated_at.
- optional metadata.

Supported interaction types should include:

- Personal rating.

- Like.
- Dislike.
- Watchlist add.
- Watchlist removal.
- Recommendation click.
- Recommendation feedback.

Implement appropriate validation and uniqueness rules to prevent accidental duplicate records.

Use clear semantics for updating or removing a watchlist item.

Provide API endpoints equivalent to:

POST /api/interactions

GET /api/interactions

GET /api/interactions/summary

GET /api/users/{user_id}/watchlist

GET /api/users/{user_id}/ratings

POST /api/recommendation-feedback

Add additional endpoints only when they serve a real use case.

Protect access to user-specific information. Do not allow one user to retrieve another user's private interaction history without authorization.

If full user registration is beyond the existing project's scope, implement a minimal, clearly documented identity mechanism suitable for the assessment. Do not pretend anonymous browser activity represents a verified real user.

#### 11. HOW DATA EXCHANGE SHOULD WORK

Implement the following end-to-end scenarios.

## Scenario A: Metadata edit

1. I open a movie in my analytics dashboard.
2. I edit its overview or another permitted field.
3. My frontend sends PATCH to the shared API.
4. The API validates and persists the change.
5. The API returns the saved record.
6. My dashboard retrieves the updated record.
7. My partner's dashboard can retrieve the same updated record.
8. Any relevant analytics are recalculated.

## Scenario B: Partner changes movie metadata

1. My partner opens a movie in their recommendation dashboard.
2. They edit a permitted field.
3. Their application submits PATCH to the same API.
4. The change is saved to the same database.
5. My dashboard retrieves the updated record and reflects the change.

## Scenario C: User rating

1. A user rates a movie in the recommendation application.
2. The partner's application submits the personal rating through POST /api/interactions.
3. The API persists the interaction.
4. My dashboard can retrieve aggregate user-interaction statistics.
5. My dashboard can show personal-rating distributions, popular genres, watchlist activity, or engagement trends based on the recorded interactions.

## Scenario D: Recommendations

1. The recommendation application fetches current movie metadata.
2. It computes recommendations using its own logic or an agreed recommendation service.
3. It records recommendation impressions, clicks, or feedback where appropriate.
4. My dashboard analyzes available recommendation-engagement metrics.

Do not claim that a recommendation was clicked or watched unless the application actually recorded that event.

Implement a reliable refresh strategy so changes appear after a refetch or refresh. WebSockets are optional, not required.

#### 12. RECOMMENDATION INTEGRATION BOUNDARY

My partner owns the recommendation application.

Do not merge the entire recommendation frontend into my analytics dashboard.

Instead:

- Keep the shared movie API reusable.
- Document the API base URL.
- Provide an example Python client for my partner.
- Provide a small example request/response integration test.
- Define the interaction schema.
- Ensure recommendation feedback can be stored through the common API.
- Explain how both applications use the same movie IDs and database.

The recommendation algorithm can remain in my partner's separate project. If a recommendation endpoint is hosted in the shared service, document exactly which component owns it.

### 13. LIVE ANALYTICS AND CACHE INVALIDATION

After metadata changes:

- Movie detail views must reflect the saved values.
- Search must use the current records.
- Top-rated rankings must use the current stored rating.
- Relevant analytics must reflect the update.
- The CSV export must reflect persisted edits.

After user interactions:

- Interaction counts must update.
- User-rating analytics must use the interaction table.
- Watchlist analytics must use actual watchlist records.
- Recommendation engagement must use actual recorded events.

If the application caches DataFrames or analytics results, invalidate or refresh the cache after writes.

Do not make the entire application dependent on a stale in-memory copy.

Keep queries efficient. Use suitable indexes and avoid reloading or reparsing the entire CSV for each request.

### 14. AUTHENTICATION AND SECURITY

Implement appropriate access control.

Requirements:

- Read-only movie browsing may be public for the demo.
- Movie edits must require editor authentication.
- User-specific interactions must be protected.
- Do not expose database credentials or privileged API keys in browser code.
- Store secrets in environment variables.
- Keep `.env` out of Git.
- Validate all incoming requests.
- Use parameterized database operations through SQLAlchemy.
- Use HTTPS in deployment.
- Restrict CORS to the actual frontend origins in production.
- Avoid logging passwords, tokens, or secrets.
- Do not allow anonymous clients to overwrite protected metadata without authorization.

If multiple editor accounts are supported, both students must be able to edit the shared catalogue using the configured permissions.

### 15. SHARED DATA PLACEMENT AND HOSTING

Use:

- Supabase PostgreSQL as the shared database.
- Render or a comparable Python host for the shared FastAPI API.
- One documented API base URL for both applications.

Both applications should be able to run independently while using the same shared service.

The local development mode should remain available.

Do not fabricate a deployed URL or claim deployment is complete without credentials.

Provide `.env.example` with variable names only. Never commit real credentials.

Provide explicit setup instructions for:

1. Creating the database.
2. Configuring the connection string.
3. Initializing the schema.
4. Importing the dataset once.
5. Creating editor accounts.
6. Deploying FastAPI.
7. Configuring both applications with the same API base URL.
8. Testing cross-application updates.

#### 16. DATA EXPORT

Implement an appropriately protected CSV export that reflects the current database.

For example:

GET /api/export/movies.csv

Generate the CSV from the current database records, including edits.

Do not simply serve the original, unchanged CSV file.

Document how to back up the database and how to restore it safely.

Keep the initial CSV files as backups.

#### 17. PYTHON-ONLY LAB ASSEMBLY

I am required to bring Python source files to my final review.

Keep the project Python-centered and preserve the existing architecture, with HTML/CSS/JavaScript embedded in Python and served by FastAPI.

Do not create a separate React or Node.js application.

Keep the source files readable and modular. Avoid unnecessary abstractions and excessive files.

Provide:

- A minimal run command.
- A documented dependency installation procedure.
- A Python-only assembly guide.
- An explanation of the order in which files depend on one another.
- A demonstration mode that can run locally using a local database.

The original CSV files may be supplied separately for initial import. Do not embed the entire dataset into Python source code.

The hosted version must use PostgreSQL, while local development can use SQLite if the existing code supports it.

#### 18. TESTING

Create and run automated tests for:

- Dataset import.
- Duplicate prevention.
- Movie listing and details.
- Search and filtering.
- Analytics.
- Metadata updates.
- Validation errors.
- Unauthorized writes.
- Persistent changes after application restart.
- Cache invalidation.
- CSV export after edits.
- Interaction creation and retrieval.
- User-rating aggregation.
- Watchlist behavior.
- Independent clients sharing the same database.
- Cross-application data consistency.

Create an integration test where:

1. Client A fetches a movie.
2. Client A edits its metadata.
3. Client B fetches the same movie.
4. Client B sees the saved update.
5. Analytics reflect the change.
6. The CSV export includes the new value.

Create a second integration test where a personal rating is submitted as an interaction and appears in the user-interaction analytics without overwriting the original aggregate movie rating.

Do not claim that tests pass unless they were actually executed.

Do not run destructive tests against production data.

#### 19. IMPLEMENTATION AND DELIVERY

Work in phases:

1. Inspect the uploaded project.
2. Identify existing working features.
3. Implement database and API improvements.
4. Implement interaction storage.
5. Enrich poster paths safely.
6. Upgrade the visual design.
7. Integrate every frontend feature with the real API.
8. Test the complete data exchange.
9. Fix failures.
10. Update the README.
11. Provide the finished project and final-review instructions.

Do not stop after producing a plan. Continue implementing in the workspace.

Do not fabricate data, API results, test results, or deployment status.

If a credential or external service is unavailable, implement the code, test what can be tested locally, and clearly state what still needs configuration.

The finished result must be visually polished, analytically useful, and demonstrably connected to the shared API and database.

Begin by inspecting the uploaded ZIP and proceed with the implementation.

# 8. API Integration

The application uses FastAPI to expose analytical data through REST API endpoints.

## API Functionality

The API supports communication between the frontend, backend, and potential external consumers.

| Endpoint | Purpose |
|---|---|
| `/api/health` | Checks application health and availability. |
| `/api/analytics/summary` | Retrieves summary analytics for the dashboard. |

The application also includes partner integration functionality through `partner_client.py`. Additional API capabilities are implemented in the backend, depending on the configured routes.

### API Request and Response Flow

1. The user opens the dashboard.
2. The frontend sends an HTTP request to the backend.
3. FastAPI receives the request and invokes the relevant functionality.
4. The analytics layer retrieves or processes the required data.
5. The backend returns a structured JSON response.
6. The frontend displays the response through charts, tables, and summary cards.

### Example API Response

The following is an illustrative example of a JSON response for a movie analytics summary:

```
{
  "total_movies": 4803,
  "average_rating": 6.2,
  "genres": [
    "Action",
    "Comedy",
    "Drama"
  ]
}
```

The values are illustrative and should not be treated as verified live output from the application.

# 9. Users, Inputs, and Outputs

### Intended Users

Movie enthusiasts: Explore movie genres, ratings, and trends.

Students and researchers: Analyze movie metadata and identify patterns in the dataset.

Data analysts: Use the dashboard to perform exploratory analysis and compare movie attributes.

External application developers: Access supported analytics through API endpoints.

### Inputs

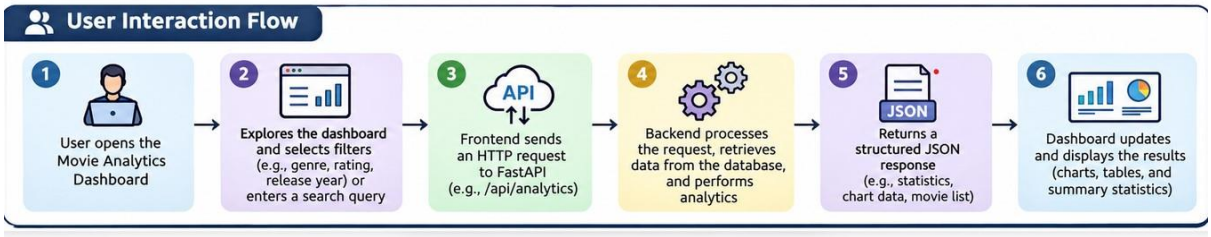
| Input | Type / Format | Purpose |
|---|---|---|
| Movie metadata | CSV | Provides movie details and attributes. |
| Movie credits | CSV | Provides cast and crew information. |
| Search query | Text | Searches for movies or relevant records. |
| Genre filter | Categorical value | Filters movies by genre. |
| Rating filter | Numeric value | Filters movies by rating. |
| Release-year filter | Integer or range | Filters movies by release year. |
| API request | HTTP / JSON | Requests analytics or movie information. |

### Outputs

The application generates:

- Interactive charts and visualizations.
- Summary statistics and KPI cards.
- Filtered movie tables.
- Genre and rating comparisons.
- Release-year trend analysis.
- Structured JSON responses through API endpoints.
- Persisted movie records in the local database.

### User Interaction Flow




# 10. References


1. Kaggle. TMDB 5000 Movie Dataset. https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata
2. The Movie Database (TMDB). Developer Documentation. https://developer.themoviedb.org/docs
3. FastAPI. FastAPI Documentation. https://fastapi.tiangolo.com/
4. Python Software Foundation. Python Documentation. https://docs.python.org/3/
5. Pandas. Pandas Documentation. https://pandas.pydata.org/docs/
6. NumPy. NumPy Documentation. https://numpy.org/doc/
7. SQLAlchemy. SQLAlchemy Documentation. https://docs.sqlalchemy.org/
8. Pydantic. Pydantic Documentation. https://docs.pydantic.dev/
9. Uvicorn. Uvicorn Documentation. https://www.uvicorn.org/
10. Pytest. Pytest Documentation. https://docs.pytest.org/
11. Render. Web Services Documentation. https://render.com/docs/web-services